

Instruction Tuning and Preference Fine-Tuning of Qwen3-0.6B

NLP with Deep Learning (Mini-Project 2) – Track 1A

Abdullah Iqbal - 26904 and Anushe Ali – 26418

[Link to Github Repository](#)

Table of Contents

<i>I. Introduction</i>	<i>3</i>
<i>II. Platform Details.....</i>	<i>3</i>
<i>III. Data Details</i>	<i>4</i>
A. The Manual Test Set of 10 Questions.	4
B. The Supervised Fine-Tuning Dataset.....	5
C. The Preference Dataset.....	5
<i>IV. Code and Architecture.....</i>	<i>6</i>
A. Repository layout.....	6
B. The Base Model and Tokenizer	6
C. How LoRA is Applied	6
D. The Training Loop	7
E. Generation and Scoring.....	7
F. Best Model Selection	7
<i>V. Experiments, Results, and Analysis.....</i>	<i>8</i>
A. Baseline.....	8
B. Supervised Fine-Tuning (Five Trials).....	8
C. Preference Fine-Tuning with DPO (Five Trials)	9
D. LoRA & Training Effects	9
E. Base vs Supervised vs Preference Tuned.....	10
F. Settings of the Best Models	11
G. Example Answers Showing the Change.....	12
H. Resource Usage and Training Time.....	12
I. Strengths, Weaknesses, and Where Each Method Fits	12
J. Common Failure Cases and Surprises	13
<i>VI. Reproducibility.....</i>	<i>13</i>
<i>VII. References.....</i>	<i>14</i>

I. Introduction

In this assignment we took a small base language model, Qwen3-0.6B-Base, and put it through a full two stage fine-tuning pipeline so that it would actually follow instructions properly. We chose Track 1 with Option A, which means we first did supervised fine-tuning and then preference fine-tuning with DPO on top of the best supervised model.

The work happened in three steps. First we measured how the raw base model behaves on a fixed set of ten questions that we wrote ourselves. Then we ran five separate supervised fine-tuning runs with LoRA, each using a different combination of adapter settings and training settings, and picked the best one. Finally we ran five DPO runs starting from that best supervised checkpoint and again picked the best. Every model at every stage was scored on the same ten questions using BLEU and BERTScore, and where two runs were too close to separate we used the lower validation loss to break the tie.

The final model improves BLEU from about 9.8 to 11.9 and BERTScore F1 from 0.880 to 0.886 compared with the untuned base model, and the difference in how the answers actually read is much larger than those numbers suggest. We ran the experiments on two free cloud platforms, Google Colab and Kaggle, and Section 1 says exactly which run was done on which platform.

II. Platform Details

We did not use any local GPU. Everything ran on free cloud notebooks. The reason two platforms appear in this report is simple: the free Colab GPU allowance ran out part way through the DPO stage, so we moved the rest of the DPO runs over to Kaggle and carried on from the same saved checkpoint. Both platforms give a Turing class T4 GPU and run the same library versions, and since every model is scored again on the exact same ten questions with the exact same scoring code, the numbers stay comparable across the two machines.

Table 1 below lists where each individual run was executed. The evidence is the working path that each saved LoRA adapter was written to, which is recorded inside the result files.

Table 1. Platform used for each stage and trial.

Stage	Trial	Platform	Saved adapter path (evidence)
Baseline	n/a	Google Colab	/content/drive/MyDrive/assignment-4
SFT	Trial 1	Google Colab	/content/drive/MyDrive/.../adapters/sft_trial1
SFT	Trial 2	Google Colab	/content/drive/MyDrive/.../adapters/sft_trial2
SFT	Trial 3 (best)	Google Colab	/content/drive/MyDrive/.../adapters/sft_trial3
SFT	Trial 4	Google Colab	/content/drive/MyDrive/.../adapters/sft_trial4
SFT	Trial 5	Google Colab	/content/drive/MyDrive/.../adapters/sft_trial5
DPO	Trial 1	Google Colab	/content/drive/MyDrive/.../adapters/dpo_trial1
DPO	Trial 2	Kaggle	/kaggle/working/assignment-4/adapters/dpo_trial2
DPO	Trial 3 (best)	Kaggle	/kaggle/working/assignment-4/adapters/dpo_trial3
DPO	Trial 4	Kaggle	/kaggle/working/assignment-4/adapters/dpo_trial4
DPO	Trial 5	Kaggle	/kaggle/working/assignment-4/adapters/dpo_trial5

To put it in one sentence: the baseline, all five supervised runs, and the first DPO run were done on Google Colab, while DPO runs two through five were done on Kaggle. The final comparison and plotting notebook was run back on Colab over the combined result files.

On the hardware side, both platforms gave us a single NVIDIA T4 with 16 GB of memory (the Colab "T4" runtime and the Kaggle "GPU T4 x1" accelerator). Because Qwen3-0.6B is so small, plain fp16 training with LoRA fits comfortably in that memory and we did not need any 4-bit or QLoRA style quantization.

The software stack was Python 3 with PyTorch, the Hugging Face transformers library, trl for the SFT and DPO trainers, peft for the LoRA adapters, datasets for loading the data, sacrebleu for BLEU, and bert_score for BERTScore using the roberta-large backbone. Mixed precision is chosen automatically at runtime: bf16 if the GPU supports it, otherwise fp16.

III. Data Details

A. The Manual Test Set of 10 Questions.

We wrote ten general instructions and question answering prompts by hand. They cover a spread of task types on purpose: plain explanation, drafting a piece of text, giving practical advice, short summarization, translation, a step-by-step procedure, a how to, and explaining a concept to a beginner. For each question we asked Claude to produce a high-quality reference answer, then we read every reference ourselves to make sure it was correct and clear before locking it in. Those Claude answers act as the gold references for both BLEU and BERTScore.

The full set lives in data/test_set.json. The same ten questions are used to score the base model, all five supervised runs, and all five DPO runs, so every stage is measured against one fixed yardstick. Table 2 lists the questions.

Table 2. The ten evaluation questions (gold references generated with Claude).

#	Question
1	Explain the difference between supervised and unsupervised learning in simple terms.
2	Write a short, polite email to a professor requesting a one-week extension on an assignment.
3	What are three practical tips for improving focus while studying?
4	Summarize the water cycle in two sentences.
5	Translate the sentence 'Knowledge is power' into French and Spanish.
6	Give me a simple recipe for making pancakes from scratch.
7	What is photosynthesis and why is it important for life on Earth?
8	List the steps to set up a basic Python virtual environment.

9	Explain what a stock market index is to someone with no finance background.
10	Describe the benefits of regular physical exercise for mental health.

B. The Supervised Fine-Tuning Dataset

For the supervised stage we used databricks/databricks-dolly-15k from Hugging Face. It is an open instruction following dataset of roughly fifteen thousand human written records, each with an instruction field, an optional context field, and a response field, spread across categories like open question answering, closed question answering, brainstorming, classification, and summarization. It is not a dataset that was used in class.

For preprocessing we kept only the examples that have no separate context, so that each training example is a clean single turn instruction and response pair, which matches the shape of our test questions. We then shuffled with seed 42 and took the first **3,000** examples for training, which we split 90/10 into roughly **2,700 training** and **300 validation** examples. Sequences were capped at 512 tokens. The reason for taking 3,000 rows rather than all fifteen thousand is that Qwen3-0.6B is a tiny model and the point of this stage is to teach it the habit and format of following an instruction, not to cram knowledge into it. Three thousand examples is more than enough to show that behavioural change, and it keeps each of the five runs inside the free GPU time we had on a single T4. Going larger would have multiplied the time across five separate runs without changing the conclusions.

Every example was wrapped in one consistent template, which we also use at evaluation time:

Instruction:
 {instruction}

Response:
 {response}

Using a single template everywhere keeps the comparison fair across the base, supervised, and DPO models. This matters because Qwen3-0.6B-Base has no built-in chat template at all, so we had to define our own.

C. The Preference Dataset

For DPO we used trl-lib/ultrafeedback_binarized from Hugging Face. For each prompt this dataset gives a chosen (preferred) response and a rejected response, which is exactly the signal DPO needs. It is the standard TRL ready form of UltraFeedback and is not a dataset used in class. We shuffled with seed 42 and took 2000 preference pairs, split 90/10 into roughly 1800 training and 200 validation pairs, with sequences capped at 512 tokens and prompts at 256. We use a slightly smaller subset than SFT because each DPO step is heavier (it processes a chosen and a rejected response and also runs a frozen reference model), and 2000 pairs was enough to produce a clear and consistent preference effect across all five runs within budget. The prompt, chosen, and rejected fields map directly onto what the TRL DPO trainer expects.

IV. Code and Architecture

This section explains how the project is put together, since the report should be enough for someone else to understand and rerun the work.

A. Repository layout

The project is organized so that each stage of the pipeline is its own notebook and the shared logic sits in one helper file.

Path	What it holds
<code>data/test_set.json</code>	The ten questions with their Claude gold references.
<code>notebooks/00_baseline.ipynb</code>	Loads the base model and scores it on the ten questions.
<code>notebooks/01_sft.ipynb</code>	Runs the five supervised LoRA trials and picks the best.
<code>notebooks/02_dpo.ipynb</code>	Runs the DPO trials from the best supervised checkpoint (Colab version).
<code>notebooks/02_dpo_kaggle.ipynb</code>	The Kaggle version of the DPO notebook used for trials two to five.
<code>notebooks/03_comparison.ipynb</code>	Builds the comparison table and the two figures.
<code>src/eval_utils.py</code>	Shared generation, scoring, and model selection helpers.
<code>results/</code>	Per trial outputs and scores in JSON, plus <code>summary.csv</code> .
<code>adapters/</code>	The saved LoRA adapters for all ten trials.
<code>figures/</code>	The two bar charts used in this report.

B. The Base Model and Tokenizer

The base model is Qwen/Qwen3-0.6B-Base, a decoder only transformer with about 0.6 billion parameters. We deliberately picked the base version rather than an already instruction tuned checkpoint. This is the most important design choice in the whole assignment. Starting from a raw base model means any ability to follow instructions has to come from our own fine-tuning, so the improvement we measure is genuinely ours and is visible in the numbers. An instruct checkpoint would already answer well and would hide the effect of what we did.

The tokenizer is the matching Qwen3 tokenizer. Since the base tokenizer has no dedicated padding token, we set the padding token to the end of sequence token.

C. How LoRA is Applied

Instead of updating all 0.6 billion weights, we use LoRA, which freezes the original model and trains small low rank adapter matrices that sit alongside chosen weight matrices. This is what makes five separate training runs affordable on a free T4. Two settings control how big and how expressive each adapter is. The rank r sets the size of the low rank update, and the list of target modules decides which weight matrices get an adapter. We used a LoRA dropout of 0.05 throughout. Across the five supervised runs we deliberately

swept these from a tiny adapter on just the attention query and value projections up to a full adapter on every linear layer in the model, so we could see how coverage and capacity affect quality. For the DPO stage we held the inner LoRA fixed (rank 16, alpha 32, on the q, k, v, and o projections) and varied only the DPO specific knobs, so that any change in the DPO results is attributable to those knobs rather than to a different adapter shape.

D. The Training Loop

Both stages reuse the same general shape, just with a different trainer from the trl library. The supervised stage uses the SFT trainer, which does ordinary next token prediction on the formatted instruction and response text. The DPO stage uses the DPO trainer, which loads the best supervised adapter as the starting policy, keeps a frozen reference copy of that same model, and then nudges the policy to prefer the chosen responses over the rejected ones. The strength of that nudge is controlled by the beta value, which is the main thing we vary in the DPO runs. A smaller beta lets the model move further away from the reference, and a larger beta keeps it closer.

E. Generation and Scoring

All evaluation uses one helper, `generate_response`, which wraps a question in our template, generates with greedy decoding (no sampling) up to 256 new tokens, and strips the prompt back off so only the model's answer is scored. Greedy decoding is deterministic, which is what makes the BLEU and BERTScore numbers reproducible from one run to the next.

Scoring is done by two functions. `compute_bleu` returns the mean sentence level sacreBLEU over the ten questions on a 0 to 100 scale, which measures word and phrase overlap with the gold answer. `compute_bertscore` returns the mean BERTScore F1 on a roughly 0 to 1 scale using roberta-large, which measures whether the meaning matches even when the wording is different.

F. Best Model Selection

The selection rule lives in `select_best`. It ranks the trials by a single combined score, $\text{composite} = 0.5 * (\text{BLEU} / 100) + 0.5 * \text{BERTScore_F1}$, which puts BLEU and BERTScore on the same scale and weighs them equally. If two trials land within a small margin of each other (0.005 on the composite), the rule treats them as tied and breaks the tie by choosing the one with the lower validation loss. This is a direct encoding of the assignment instruction to use BLEU plus BERTScore first and validation loss as the tie breaker.

V. Experiments, Results, and Analysis

A. Baseline

Table 3 shows how the untuned base model does on the ten questions.

Table 3. Base model results on the ten question test set.

Model	BLEU	BERTScore F1	Composite
Qwen3-0.6B-Base	9.82	0.8803	0.4892

The base model already produces text that is semantically in the right area, which is why BERTScore is fairly high at 0.88, but it is weak at actually doing what it is told. It often repeats the question back, slips into meta commentary about the task, or loops on a phrase. Examples of this are in Section 4.6 and 4.9.

B. Supervised Fine-Tuning (Five Trials)

Each supervised run changed both the LoRA settings (rank, scaling alpha, and which modules get an adapter) and the training settings (learning rate, batch size, epochs, and gradient accumulation where used). Taken together the five runs cover a wide span, from a small attention only adapter in Trial 1 to a full all linear adapter in Trial 5. All five supervised runs were done on Google Colab.

Table 4. Supervised trial settings and results, all run on Colab.

Trial	LoRA r	alpha	Target modules	LR	Batch	Grad accum	Epochs	BLEU	BERTScore F1	Val loss
1	8	16	q, v	2e-4	8	1	1	6.12	0.8752	n/a
2	16	32	q, k, v, o	2e-4	8	1	2	6.12	0.8785	n/a
3	32	64	q, k, v, o, gate, up, down	1e-4	4	1	2	9.47	0.8836	n/a
4	16	32	q, v	3e-4	4	4	3	6.57	0.8755	2.125
5	64	128	all-linear	5e-5	2	2	1	5.77	0.8721	2.108

We logged validation loss for the later runs (4 and 5). It was not needed to pick the winner here because Trial 3 is the clear best on both BLEU and BERTScore, so the tie breaker never came into play.

The best supervised model is Trial 3. As Table 4 shows, it has the highest BLEU at 9.47 and the highest BERTScore at 0.8836 of the five, so it wins outright on the combined score. What makes Trial 3 work is that it adapts the most useful set of weights, namely all the attention projections plus the MLP gate, up, and

down projections, at a sensible middle rank of 32 and a calm learning rate of $1e-4$ over two epochs. The adapter is saved at `adapters/sft_trial3` and becomes the starting point for the DPO stage.

C. Preference Fine-Tuning with DPO (Five Trials)

Starting from the best supervised adapter, we ran five DPO trials, varying beta, learning rate, gradient accumulation, and epochs. Trial 1 was done on Colab and trials 2 through 5 were done on Kaggle after the Colab GPU quota ran out.

Table 5. DPO trial settings and results.

Trial	beta	LR	Batch	Grad accum	Epochs	BLEU	BERTScore F1	Val loss	Platform
1	0.10	5e-5	2	2	1	9.91	0.8854	0.6465	Colab
2	0.10	1e-4	2	1	2	10.07	0.8807	0.8180	Kaggle
3	0.05	5e-5	2	2	1	11.94	0.8863	0.6389	Kaggle
4	0.30	5e-5	2	2	2	10.74	0.8891	0.8553	Kaggle
5	0.10	2e-5	2	4	1	11.61	0.8893	0.6463	Kaggle

The best combined model is DPO Trial 3, and it is worth spelling out why, because the two metrics do not point at exactly the same run. On BLEU, Trial 3 is clearly ahead at 11.94, with Trial 5 next at 11.61. On BERTScore the top three are almost level: Trial 5 at 0.8893, Trial 4 at 0.8891, and Trial 3 at 0.8863, a gap of only about 0.003. On the combined score, Trials 3 and 5 sit inside the small tie margin, so the lower validation loss decides it. Trial 3 has the lowest validation loss of all the DPO runs at 0.6389, just under Trial 5 at 0.6463 and Trial 1 at 0.6465. So Trial 3 wins on BLEU outright and also wins the close BERTScore race through the validation loss tie breaker. The thing that defines Trial 3 is its low beta of 0.05, which lets the policy move further toward the preferred answers and gave the biggest jump in BLEU. The adapter is saved at `adapters/dpo_trial3`.

D. LoRA & Training Effects

From Tables 4 and 5, clear patterns are seen. The first is that which modules you adapt matters more than the raw rank. The two supervised runs that only touched attention (Trials 1 and 2) got stuck around a BLEU of 6.1 even with a higher rank. The jump to 9.47 only happened in Trial 3 once the MLP gate, up, and down projections were added. Adapting the feed forward layers, not just attention, is what taught the model to actually follow the instruction format.

The second is that a bigger adapter is not automatically better. Trial 5 used the largest setup, rank 64 on every linear layer, but trained it at the lowest learning rate for only one epoch, and it came out worst at a BLEU of 5.77. A high capacity adapter that barely gets trained underfits, so the capacity is wasted without enough optimization steps at a reasonable learning rate.

The third is that learning rate is the most delicate knob. Trial 4's aggressive 3e-4 over three epochs did not help and tended to hurt fluency, while Trial 3's calmer 1e-4 over two epochs was the sweet spot. Too high a rate destabilizes a 0.6 billion parameter model and too low a rate underfits it.

For DPO, beta is the dominant lever. Lowering beta from 0.30 in Trial 4 to 0.05 in Trial 3 moved BLEU from 10.74 to 11.94. A smaller beta relaxes the pull back toward the reference policy and allows larger preference driven shifts, which helped here because the supervised model still had room to become more aligned. The trade off is that pushing beta too low risks drifting away from clean text, which we touch on in Section 4.9.

E. Base vs Supervised vs Preference Tuned

Table 6 below shows the side-by-side comparison of the best model from each stage.

Table 6. Best model at each stage on the ten question test set.

Stage	Model	BLEU	BERTScore F1
Base	Qwen3-0.6B-Base	9.82	0.8803
Instruction tuned (best SFT)	SFT Trial 3	9.47	0.8836
Preference tuned (best DPO)	SFT plus DPO Trial 3	11.94	0.8863

These results are represented as Figure 1 (BLEU by stage) and Figure 2 (BERTScore F1 by stage) below.

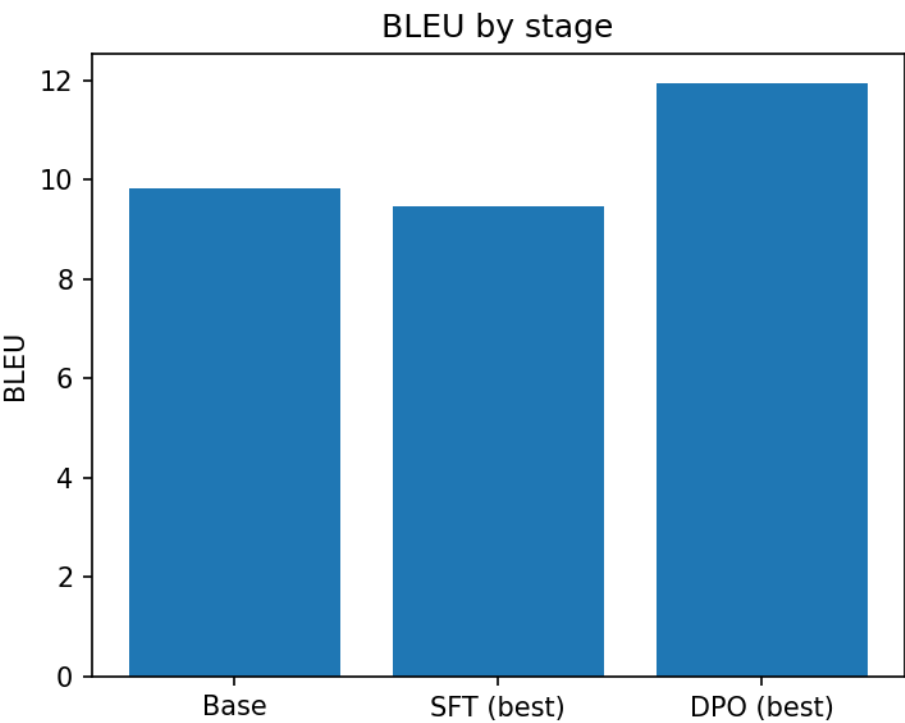


Figure 1. BLEU score by pipeline stage (Base / SFT / DPO).

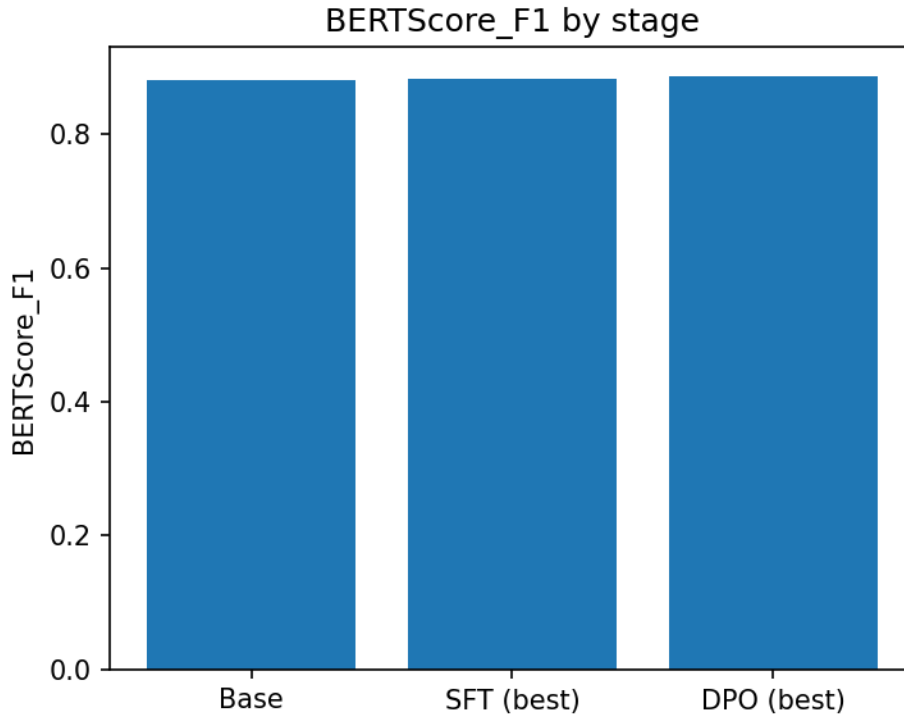


Figure 2. BERTScore F1 by pipeline stage (Base / SFT / DPO).

Going from base to supervised, BERTScore rises from 0.8803 to 0.8836 while BLEU actually dips slightly from 9.82 to 9.47. This looks odd at first but it is informative. The raw base model sometimes happens to echo phrasing that looks like the reference, which props up BLEU, but it does not reliably do the task. Supervised tuning makes the answers more on topic and better structured, which BERTScore rewards, even though the exact word overlap has not jumped yet. In other words the main thing supervised tuning buys you is behaviour, following the instruction and laying the answer out properly, and BLEU undercredits that.

Going from supervised to DPO is where the surface quality jumps. BLEU climbs sharply from 9.47 to 11.94 and BERTScore also improves to 0.8863. Preference tuning pushes the model toward the kind of complete, well structured answers that people prefer, which also happen to line up better with the gold references.

Across the whole pipeline, from base to final, BLEU goes up by about 2.1 points and BERTScore by about 0.006, and the clearest gains show up in instruction following, formatting like proper email layout and numbered steps, and a drop in the repetition problems.

F. Settings of the Best Models

The best supervised model, Trial 3, used LoRA rank 32 with alpha 64, adapters on the q, k, v, o, gate, up, and down projections, a learning rate of 1e-4, batch size 4, and two epochs.

The best DPO model, Trial 3, used beta 0.05, learning rate 5e-5, batch size 2, gradient accumulation 2, and one epoch, applied on top of the best supervised adapter.

G. Example Answers Showing the Change

These examples are taken straight from the per trial result files and show both the gains and a failure case.

For Question 2, the request to write a polite email asking for an extension, the base model does not actually write the email. It continues the prompt instead, printing an "Instructions" checklist and an "Example" header, treating the request as text to elaborate rather than a job to do. This is the classic base model habit. The supervised Trial 3 model produces a real email with a subject line, a greeting, a body that makes the request, and a sign off, so it gets the task and the format right. The DPO Trial 3 model produces a polished, well structured email with a clear subject, a professional tone, a direct request, and a courteous close, very close in style to the gold reference.

For Question 9, explaining a stock market index to a beginner, the base model gives a generic and slightly circular definition. The supervised Trial 3 model adds a concrete and correct example, the S&P 500 tracking 500 large U.S. companies, so it is more grounded. The DPO Trial 3 model adds an intuitive analogy (comparing an index to a barometer measuring the overall health of the weather), the S&P 500 example, and a clear explanation of market cap weighting, which is the closest match to the reference, since the reference itself uses a class average analogy. This shows DPO favouring fuller, more explanatory answers.

For Question 6, the pancake recipe, quality is not always better at each step, and this is a useful failure case. Supervised Trial 2 falls into a repetition loop, repeating "Add 1/2 cup of milk and 1/2 cup of milk to the bowl and mix" many times, which is a known small model failure. DPO Trial 4, which used a high beta, hallucinates cocoa powder into a plain pancake recipe. Even the winning DPO Trial 3 lists sugar twice with odd gram amounts, which shows there is still some factual sloppiness on recipe style content.

H. Resource Usage and Training Time

Each run used a single NVIDIA T4 with 16 GB of memory on either Colab or Kaggle, training in fp16 with LoRA and no quantization. Because we only ever trained the small LoRA adapters and not full copies of the model, each saved adapter is only a few megabytes, which is what keeps both the training time and the storage small. In practice each supervised run finished in the order of a few minutes to a few tens of minutes on a T4, with the three epoch Trial 4 being the longest, and each DPO run was similar but a bit heavier per step because DPO has to run both the policy and a reference model at once. Peak memory stayed well within the 16 GB budget for every run. The reason the work is split across two platforms is exactly this budget: the free Colab GPU quota ran out after the five supervised runs and the first DPO run, so DPO trials two to five were moved to Kaggle.

I. Strengths, Weaknesses, and Where Each Method Fits

Supervised fine-tuning is what teaches the model the basic shape of following an instruction, and it is cheap and stable. The single biggest jump in quality, from a base model that rambles to something that behaves like an assistant, happens here. Its limitation is that it only imitates the reference answers, so it cannot easily learn what makes one valid answer better than another, and a surface metric like BLEU can even dip while the behaviour clearly improves.

DPO directly optimizes for preferred answers over rejected ones without needing a separate reward model, and it refines quality, completeness, and alignment. It gave us the biggest BLEU gain and also nudged BERTScore up. Its weakness is that it is very sensitive to beta. Too high and the model barely moves, as in Trial 4, and too low and it risks drifting toward verbose or messy text. It also needs a good supervised starting point, since running DPO straight on a base model that cannot follow instructions would give little useful signal.

So the natural split is to use supervised tuning to install the core instruction following and formatting, then use DPO afterwards to polish toward the answers people actually prefer. The order matters, since DPO builds on the behaviour that supervised tuning put in place.

J. Common Failure Cases and Surprises

A few failure modes showed up repeatedly. The most common is repetition, where outputs from the base model and some of the tuned runs fall into a repeated phrase loop under greedy decoding, which is typical for small models. The second is prompt echoing, where the base model continues or restates the instruction instead of carrying it out, which is the exact thing supervised tuning fixes. The third is hallucinated or inconsistent detail on procedural content, like the recipes picking up wrong ingredients or duplicate quantities, which shows that factual reliability lags behind format and fluency for a model this size.

There were two surprises worth noting. One is that BLEU went down slightly from base to supervised even though the model clearly got better at following instructions, which is exactly why we report BERTScore alongside BLEU and select on the combined score rather than BLEU alone. The other is that low beta DPO sometimes produced answers a bit longer than the references, which helps BERTScore but is a minor mismatch for BLEU.

VI. Reproducibility

The complete project, with all notebooks, the shared utilities, the result files, the figures, and the saved LoRA adapters, is available [here](#).

To rerun the work, execute the notebooks in order: 00_baseline, then 01_sft, then 02_dpo, then 03_comparison. The file 02_dpo_kaggle.ipynb is the Kaggle path version of the DPO notebook used for trials two to five.

The base model is Qwen/Qwen3-0.6B-Base, the supervised data is databricks/databricks-dolly-15k (3,000 no-context examples, 90/10 split, max length 512), and the preference data is trl-lib/ultrafeedback_binarized (2,000 pairs, 90/10 split, max length 512 and max prompt length 256), all from Hugging Face. The one prompt template shown in Section 2.2 is used for both training and evaluation across all stages. LoRA dropout was 0.05 everywhere, and the DPO stage used a fixed inner LoRA of rank 16, alpha 32, on the q, k, v, and o projections. We fixed seed 42 for every dataset shuffle and for the 90/10 train and validation splits, and we used greedy decoding with no sampling for all evaluation, so the BLEU and BERTScore numbers reproduce.

BLEU is computed with sacrebleu as the mean sentence level score, BERTScore F1 is computed with bert_score using roberta-large, and the selection composite is $0.5 * \text{BLEU}/100 + 0.5 * \text{BERTScore_F1}$ with a lower validation loss tie breaker. Every trial setting is listed in Tables 4 and 5, the raw per question outputs and scores are in the results folder (baseline.json, sft_trials.json, sft_trial1 through sft_trial5, dpo_trials.json, dpo_trial1 through dpo_trial5, and summary.csv), and the trained adapters for all ten trials are saved under adapters. As a reminder of platforms, the baseline, all supervised runs, and the first DPO run were on Colab, and DPO runs two to five were on Kaggle. The results are platform independent because every model is scored again on the same fixed test set with the same scoring code.

VII. References

1. Anthropic. Claude (used to generate the gold reference answers for the ten question test set). <https://claude.ai>
2. Qwen Team. Qwen3-0.6B-Base model card, Hugging Face. <https://huggingface.co/Qwen/Qwen3-0.6B-Base>
3. Databricks. databricks-dolly-15k dataset card, Hugging Face. <https://huggingface.co/datasets/databricks/databricks-dolly-15k>
4. TRL and Hugging Face. ultrafeedback_binarized dataset card, Hugging Face. https://huggingface.co/datasets/trl-lib/ultrafeedback_binarized
5. Rafailov et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS 2023. <https://arxiv.org/abs/2305.18290>
6. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022. <https://arxiv.org/abs/2106.09685>
7. von Werra et al. TRL: Transformer Reinforcement Learning. <https://github.com/huggingface/trl>
8. Mangrulkar et al. PEFT: Parameter-Efficient Fine-Tuning. <https://github.com/huggingface/peft>
9. Zhang et al. BERTScore: Evaluating Text Generation with BERT. ICLR 2020. <https://arxiv.org/abs/1904.09675>
10. Post, M. A Call for Clarity in Reporting BLEU Scores (sacreBLEU). WMT 2018. <https://github.com/mjpost/sacrebleu>